

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA14

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS: 5

**Duration :60 Mins
On Class
Total Marks:50**

Annexure A

ANALYTICAL PROBLEM SOLVING

Code of Conduct:

I, A K Selva Prabhu (192421224), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Signature of the student: A.K. Selva

Analytical problem Solving

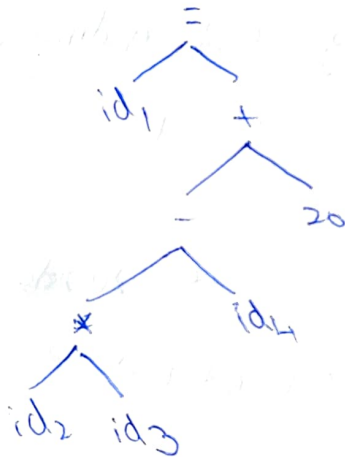
17. Apply your understanding of compiler design to demonstrate how a source is processed through all phases of compiler.

$$W = X * Y - Z + 20$$

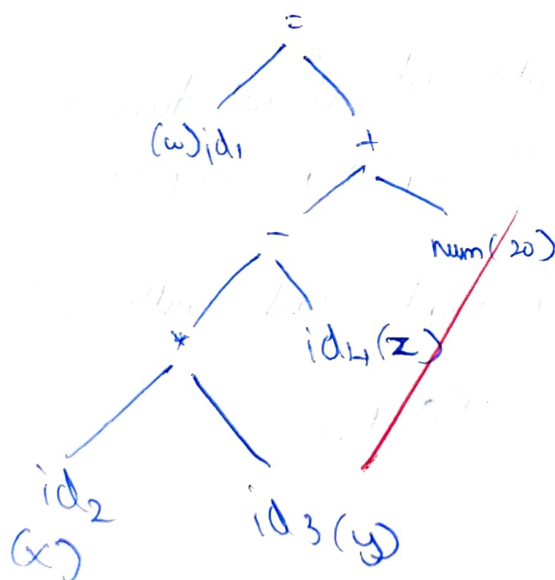
(i) Lexical Analyzer:

$\langle id_1 \rangle \langle = \rangle \langle id_2 \rangle \langle * \rangle \langle id_3 \rangle \langle - \rangle \langle id_4 \rangle \langle + \rangle \langle 20 \rangle$
num.

(ii) Syntax Analysis.



(iii) Semantic Analysis



(iv) Intermediate code Generation:

$$T_1 = id_2 * id_3$$

$$T_2 = T_1 - id_4$$

$$T_3 = 20 \text{ (num)}$$

$$T_4 = T_2 + T_3$$

$$T_4 = id_1$$

(v) Code optimization

$$t_1 = id_2 * id_3$$

$$t_2 = t_1 - id_4$$

$$t_3 = t_2 + 20$$

$$t_4 = id_1 \Rightarrow id_1 = t_3$$

v) Target code Generation:

LOAD R₁, x

LOAD R₂, y

MUL R₁, R₂

LOAD R₃, z

SUB

2) Construct regular expressions for the following subsets $\{0, 1\}^*$

(i) All strings beginning and ending with 1

$$R = 1 + 1(0+1)^*1$$

(ii) All strings containing the substring 01

$$R = (0+1)^*01(0+1)^*$$

(iii) All strings with exactly one "0".

$$R = 1^*01^*$$

(iv) All strings with at least two 1's

$$R = (0+1)^*1(0+1)^*1(0+1)^*$$

v) All strings that do not end with "0"

$$R = \Sigma + (0+1)^*1$$

93).

(i) $(\epsilon + aa^*)^*$

Consider the language and describe the possible forms of strings generated.

(ii) $(\epsilon + aa^*)^*$

$$= (a^*)^*$$

$$= a^*$$

$$L = \{ \epsilon, a, aa, aaa, \dots \}$$

(iii) $(a^* + b^*)^* abb$

$$= (a+b)^* abb$$

$$L = \{ abb, aa bb, babb, aaabb, \dots \}$$

(iv) $(a^* b^*)^* aba$

$$= (a^* b^*)^* aba$$

$$L = \{ aba, aaba, bbaba, aaaba, \dots \}$$

(v) $(aba)^* + (a^* b^*)^*$

$$= (aba)^* + (ab)^*$$

$$L = \{ \epsilon, aba, abaaba, aabbb, \dots \}$$

(vi) $(a+ab)^*$

$$L = \{ \epsilon, a, ab, aa, aab, abab, \dots \}$$

$$5) (0^* 1^*)^* = (0+1)^*$$

$$(0^* 1^*)^* = \{ \epsilon, 0, 1, 00, 01, 10, 11, \dots \}$$

$$(0+1)^* = \{ \epsilon, 0, 1, 00, 01, 10, 11, \dots \}$$

$$(0^* 1^*) = (0+1)^*$$

$$ii) (0+1)^* (0+1)^* = (0+1)^*$$

$$R^* R^* = R^*$$

$$iii) (0^*)^* = 0^*$$

$$(R^*)^* = R^*$$

$$(iv) (0+1)^* 0 (0+1)^* + (0+1)^* 1 (0+1)^* = (0+1)^*$$

$$(0+1)^* 0 + (0+1)^* 1 = (0+1)^*$$

$$(0+1)^* (0+1) = (0+1)^*$$

$$(0+1)^* = (0+1)^*$$

$$(v) \epsilon + 0(0)^* = 0^*$$

$$\epsilon + R(R^*) = R^*$$

4)

(i) $[a-zA-Z][a-zA-Z0-9]^*$

↓
Identifier
must start
with letter

↓
This allows for zero or
more subsequent letters or
digits.

Exe Rule:

/* */

if { return if: Y

else { return Else: Y

while { return while: Y

$A-zA-Z)[A-Za-z0-9]^*$ { return identifier: }

$[0-9]^+ \{ \text{return inter: } \}$

/* */

(ii) Justification.

Valid integers

0

10

20

456

9999

